

Harness 实践：让 Agent 全自动制作知识讲解视频

—— web-video-presentation Skill 完整实战解析

整理自 B 站「code 秘密花园」视频教程

从 Harness 六部分到 Claude Code 多 Agent 协作，手把手讲透完整 workflow

一、为什么不用视频生成模型，而要「折腾网页」

视频作者（花园老师）一上来就交了底：他这些精美的知识讲解视频，并不是用视频生成模型做的，也没有用 NotebookLM，而是自己 web coding 出来的网页。

原因只有两个字：可控。

在网页里，字体、配色、每一步停留几秒、某一帧要不要出现精确数字，这些都可以通过改几行代码直接搞定。相比视频模型「抽卡」式生成，网页方案更稳定、成本也更低。



图 1-1 NotebookLM 只能做静态图，而网页可以自由做动画和精确控制

二、网页效果：每个像素都能调

用网页做视频，核心优势是「逐像素可调」。

- 字体：可以随意选择和切换字体样式；
- 配色：主题色、强调色、背景色完全可控；
- 每步停留：精确控制每一步画面停留的时间（如 2.4 秒）；
- 精确帧：某一帧出现什么内容、编号是多少，都可以精确指定。

作者也试过 Remotion 这类框架，但觉得它限制了模型发挥，不如直接让模型写网页更自由。



图 2-1 网页里的字体、配色、停留时间、精确帧都能逐像素调整

三、把文章变成视频：四个关键点

想把一篇文章自动变成高质量知识讲解视频，不是把文字糊到页面上就完事。视频里总结了四个关键点：

- 把文章变成口播稿：原始技术文章通常偏书面化，要改成短句、口语化、第二人称，像聊天一样；
- 把口播稿拆成开发大纲：每一句话对应一个具体的画面步骤，每几个步骤组成一个章节，每个章节聚焦一个话题；
- 给每一步做视觉演示：尽量让网页把抽象概念用动态效果演示出来，而不是直接堆文字；
- 关键步骤跟随口播节奏：视觉演示的切换节奏必须和口播稿对齐，视频才会自然。

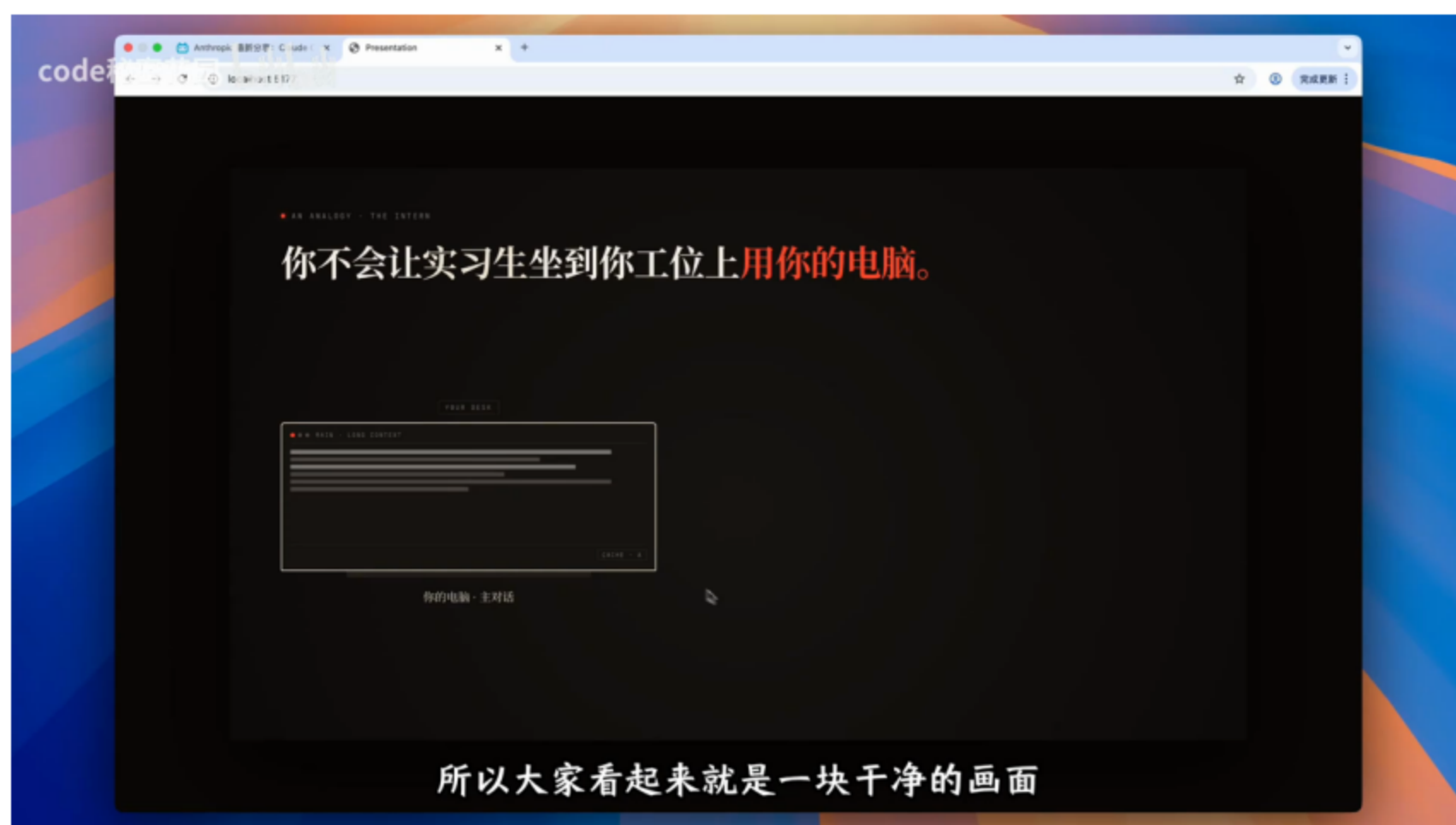


图 3-1 文章→视频必须经历的四个关键步骤：口播稿、开发大纲、视觉演示、节奏对齐

四、Skill 是给 Agent 看的「操作手册」

模型本身已经具备写文章、写代码、做音频的能力。但直接把任务丢给 Agent 说「帮我做个视频网页」，结果可能很不稳定：稿子和画面可能对不上、章节和音频数不一致、后面章节风格突然跑偏。

所以，我们需要一套机制来驾驭这个过程：

- 划定边界：告诉 Agent 什么能做、什么不能做；
- 设定检查点：让 Agent 在关键节点停下来等人确认；
- 自检和修复：每个阶段结束后必须自检，不过关就返工。

这套机制就是 Harness（驾驭工程）。而 Skill 就是一份给 Agent 看的操作手册，Agent 加载 Skill 后会按规则干活。本视频讲的 Skill 叫 web-video-presentation。



图 4-1 Skill 本质是给 Agent 加载规则：AI 加载 Skill 后按规则干活

五、从 Harness 六部分拆解这个 Skill 的设计

视频里提到，一个成熟的 Harness 一般包含六个部分。作者用这六部分拆解了 web-video-presentation 的核心设计。

Harness 部分	解决什么问题	Skill 中的实践
执行和编排	让模型知道下一步该做什么	从文章→视频切成 4 个阶段，中间卡 2 个人工检查点
状态和记忆	跨多步骤保持连续和稳定	开发计划 outline.md 作为持续记忆；口播稿+原文章+大纲三文档分工
工具系统	模型到底能做什么	用好 read_file / write_file 等文件读写工具；多章节并行时物理隔离
约束和恢复	出错了怎么办	反馈修复的最小切片：只改节奏/视觉/代码的最小单位，不返工整章
评估和观测	怎么知道自己做得对不对	硬性自检规则：自检→修复→再汇报；可用 reviewer

		agent 独立质检
信息拆分	避免上下文被无关信息稀释	6 份文档，每份只在指定阶段读取

这里也要破除一个误区：做 Harness 不等于从零做一个 Agent。用一个 Skill 约束 Agent，把垂直工作做好，同样是在做 Harness。

六、四阶段流程与两个人工检查点

整个从文章到视频的生产流程被拆成四个阶段，中间卡了两个人工检查点：

- 阶段一：内容编写。AI 同时产出「口播稿 script.md」和「开发大纲 outline.md」；
- 检查点 1（Checkpoint Plan）：确认稿子、开发计划、主题风格、素材准备方式、后续开发模式；
- 阶段二：网页开发。先做好第一章，人工验收作为后续章节基准；后续可以顺序开发或并行开发；
- 检查点 2（Checkpoint Audio）：确认是否自动合成音频；
- 阶段三：音频合成。从口播稿提取音频清单，调用 TTS 合成；
- 阶段四：录屏。浏览器打开网页，开启音频自动播放，直接录屏得到成品视频。



图 6-1 四阶段流程：内容编写 → 网页开发 → 音频合成 → 录屏，中间卡两个人工检查点

七、信息拆分：每份文档只在指定阶段读取

整个 Skill 的链路横跨四个阶段，涉及大量复杂流程。如果启动时就把所有文档一股脑灌给模型，模型的注意力会被严重稀释。

解决方案是：把所有信息拆成多份文档，每份文档只在指定阶段读取。

- 口播稿规范：只在第一阶段读取；
- 章节开发指引：只在开发阶段读取；
- 音频合成文档：只在音频合成阶段读取。

这样做的好处是：模型在任何时候面前摆的都是它最需要的信息，不会被无关内容干扰。



图 7-1 信息拆分：把资料拆成多份文档，分阶段读取，避免上下文稀释

八、状态与记忆：开发计划是最核心的记忆

当项目有十几个章节、上百个步骤时，Agent 写到后面章节，很容易忘记前面定了什么结构、用户在检查点改过什么方向。

因此，开发计划 `outline.md` 是整套系统里最重要的「记忆」。

在真正的开发阶段，Agent 还会被强制读取两个文件：

- `script.md`（口播稿）：负责整体节奏，先讲什么、后讲什么；

- article.md（原文章）：负责信息密度，口播稿精简后，画面需要的数据案例可以从原文补。

三份文档各有分工：outline.md 管结构，script.md 管节奏，article.md 管信息。Agent 同时参考它们，就能避免「结构对了但画面空了」或「信息很多但节奏乱了」的问题。本质上，这就是文件化的工作记忆。



图 8-1 outline.md 是贯穿整个项目的工作记忆，三份文档各有分工

九、工具系统：并行开发时的隔离机制

这个 Skill 没有用什么特殊工具，核心就是把 Agent 自带的 read_file / write_file 用好。

但在多章节并行开发时，关键问题是：多个 Agent 同时改一个项目，会不会冲突？

Skill 设计了严格的隔离机制：

- 独立文件夹：每张一章都是独立的文件夹，物理隔离；
- 独立 CSS 前缀：避免类名冲突；
- 主题 Token 兜底：字体、颜色、间距等视觉变量走全局变量，确保风格统一。

即使多个 Agent 独立开发，也能保持整体视觉一致、互不干扰。



图 9-1 并行开发隔离机制：独立文件夹、独立 CSS 前缀、主题 Token 统一视觉

十、约束与恢复：反馈修复的「最小切片」

当用户觉得某个章节有问题时（比如节奏太快、动画太花），Agent 的本能反应可能是「重做整个章节」。但问题是：重做会把已经做对的部分也改掉。

Skill 里的关键原则是：反馈修复的最小切片。

先定位问题在哪一层：是节奏？是视觉内容？还是代码？然后只改最小的切片，不重新做整章节。

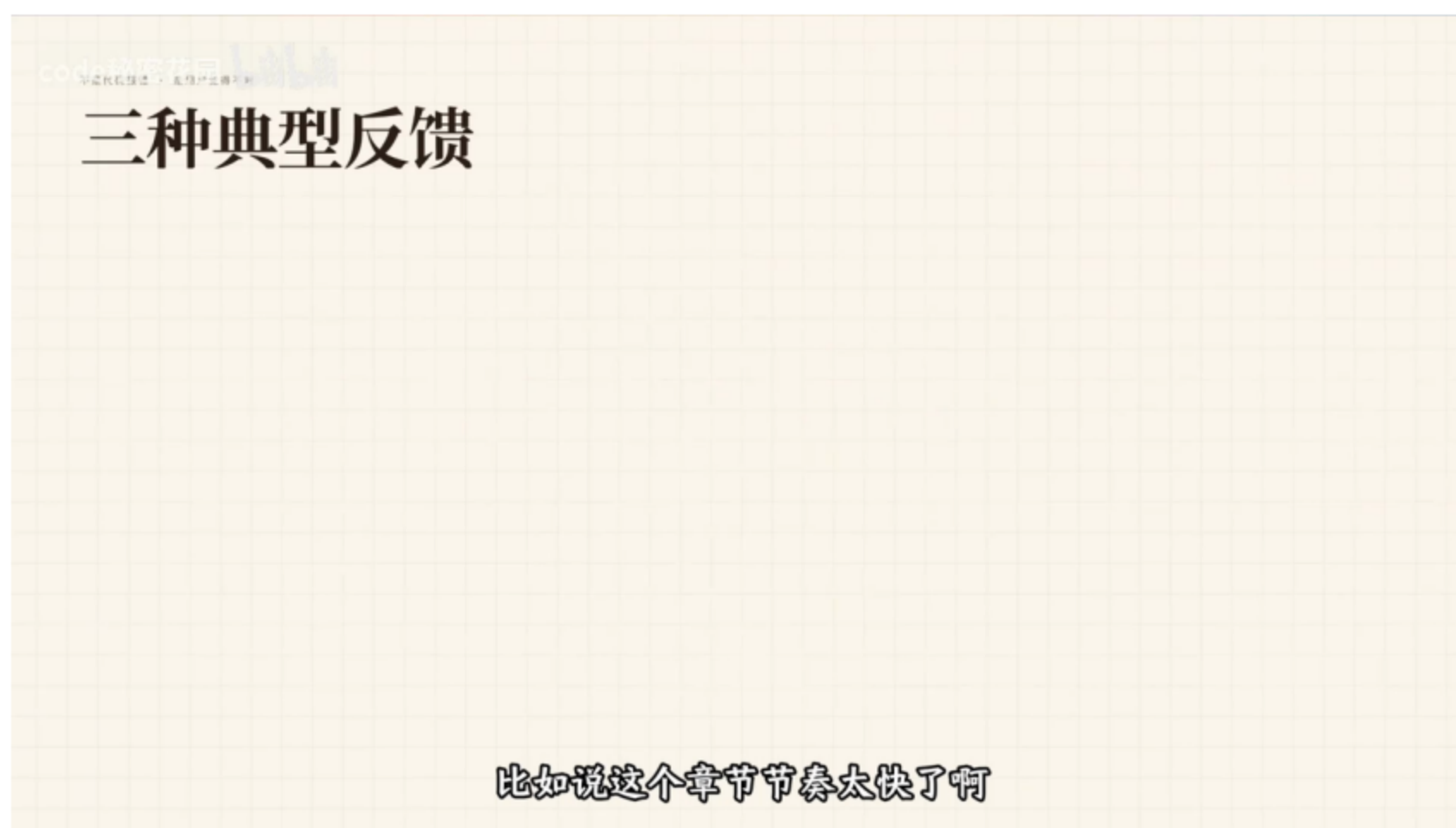


图 10-1 面对用户反馈，只改最小切片：节奏 / 视觉 / 代码，不返工整章

十一、评估与观测：解决自评失真

视频作者强调，评估和观测是他花心思最多的一块，因为它直接对应 Anthropic 提出的「自评失真」问题：让写代码的 Agent 自己评价这章写得怎么样，结果大概率是「还不错」。

因此，Skill 设计了一套非常硬性的自检规则：

- 每个关键流程完成后，必须走「自检 → 修复 → 再汇报」的流程；
- 每个阶段都有一套完整的自检清单；
- 最优方案：用 Agent Teams 开一个独立的 reviewer agent，从零开始逐项检查；
- 次优方案：用 sub agent 检查；最次也要当前 Agent 逐项核查，不允许「目测一遍就放行」。

只有质检过关，才能进入下一阶段。

十二、环境准备：四个工具

视频里用到的核心工具如下：

- Claude Code：核心执行 Agent，也可以用 Cursor、Codex 等支持 Skill 的 Agent 替代；
- MiniMax：国产模型，Token Plan 适配 Claude Code，速度快、量大、性价比高，且附带多模态套餐（含语音合成）；
- cc-switch：桌面配置工具，用于给 Claude Code 配置 MiniMax 等自定义模型；
- MiniMax CLI：官方提供，Agent 可直接调用完成语音合成，无需写代码。

Claude Code 启动时建议加上 `--dangerously-cape-permissions` 参数（会跳过工具调用的人工确认），演示时方便，但正式使用要注意安全。

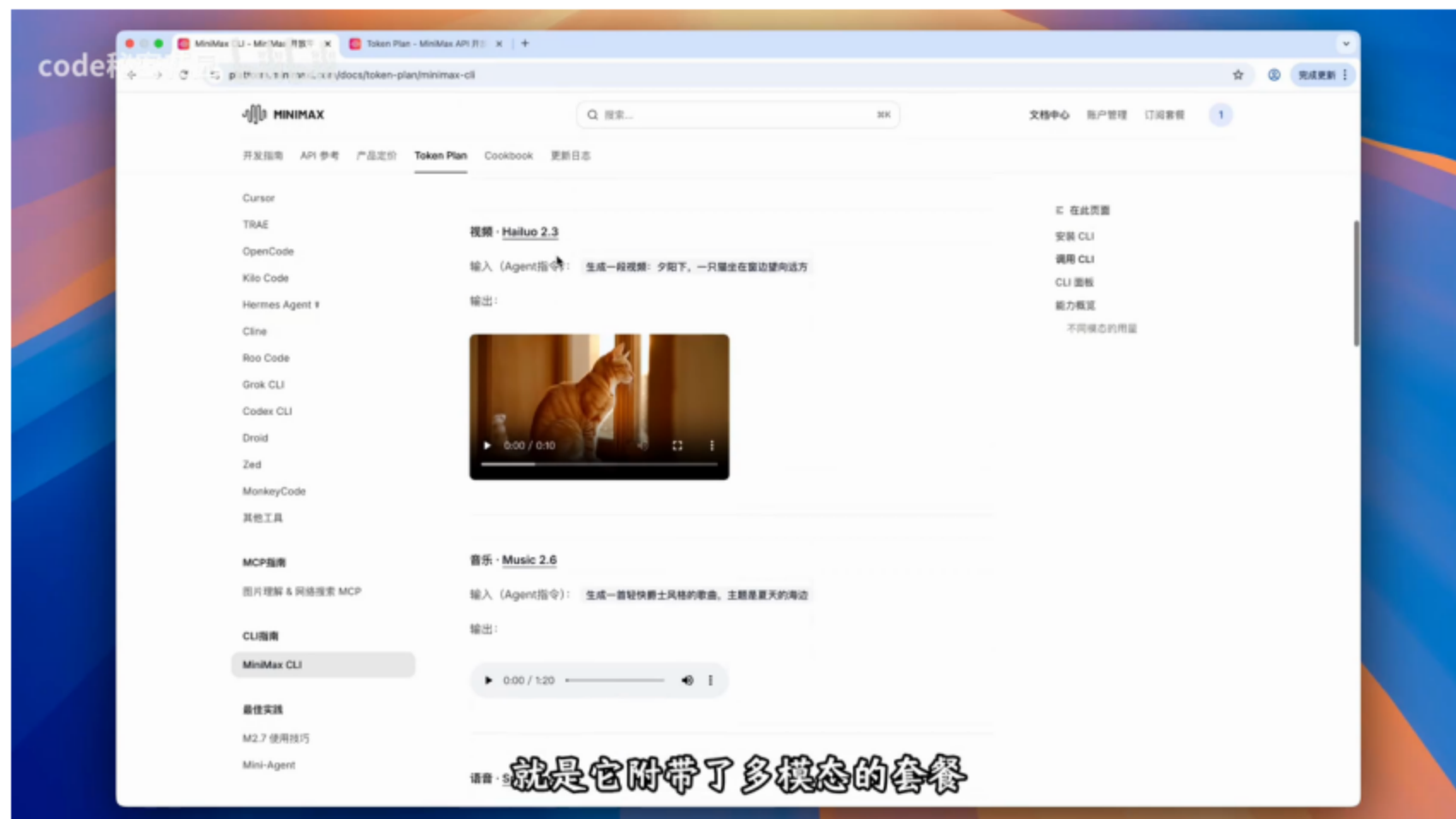


图 12-1 MiniMax CLI 提供多模态能力，含语音合成，Agent 可直接调用



图 12-2 Claude Code 启动后配置 MiniMax 模型并测试对话

十三、多 Agent 协作：SubAgent vs Agent Teams

对于十几个章节、上百个步骤的项目，让一个 Agent 从头写到尾会非常慢。Skill 在设计上支持多 Agent 并行开发。Claude Code 提供两种多 Agent 能力：

13.1 SubAgent

SubAgent 像主 Agent 把任务分派给子 Agent，子 Agent 在独立上下文里完成后再把结果交回来。各个子 Agent 互相看不到、不讨论。

- 适合：结果导向型任务，如并行开发多个章节、一次性检查某段代码；
- 优点：省 Token、容易调度。



图 13-1 SubAgent 模式：三个子 Agent 各写一章，结果回收后汇总

13.2 Agent Teams

Agent Teams 像一个项目小组：一个组长加几个组员，每个组员都是独立会话，他们之间可以互相发消息、共享任务列表。

- 适合：需要来回反馈的任务，如质检环节（测试员发现字号小 → 让开发改 → 测试员复查）；
- 代价：每个组员都是完整会话，Token 开销更高。

配合 tmux 等终端管理器，可以在一个窗口里同时看到每个组员在哪个面板工作，可视化效果更好。



图 13-2 tmux 终端管理器：在一个窗口中同时查看多个 Agent 会话



图 13-3 tmux 分屏展示多 Agent 实际运行：script reviewer 与 outline reviewer 并行质检

十四、阶段一：内容编写与多 Agent 质检

实战演示用了一篇文章《一封邮件发出后的 600ms》。进入项目目录后启动 Claude Code，输入文章并触发 Skill 的第一步。

Agent 会同时做两件事：

- 生成口播稿 script.md：把书面化文章改成适合视频朗读的口语化脚本；
- 生成开发大纲 outline.md：把文章拆成细粒度的章节和步骤，标注每一步屏幕上该出现什么。

为了让质量更稳，作者还开了两个审核 Agent（script reviewer、outline reviewer）分别对口播稿和开发大纲进行独立质检。主 Agent 根据评审结论修改，再输出最终版本。

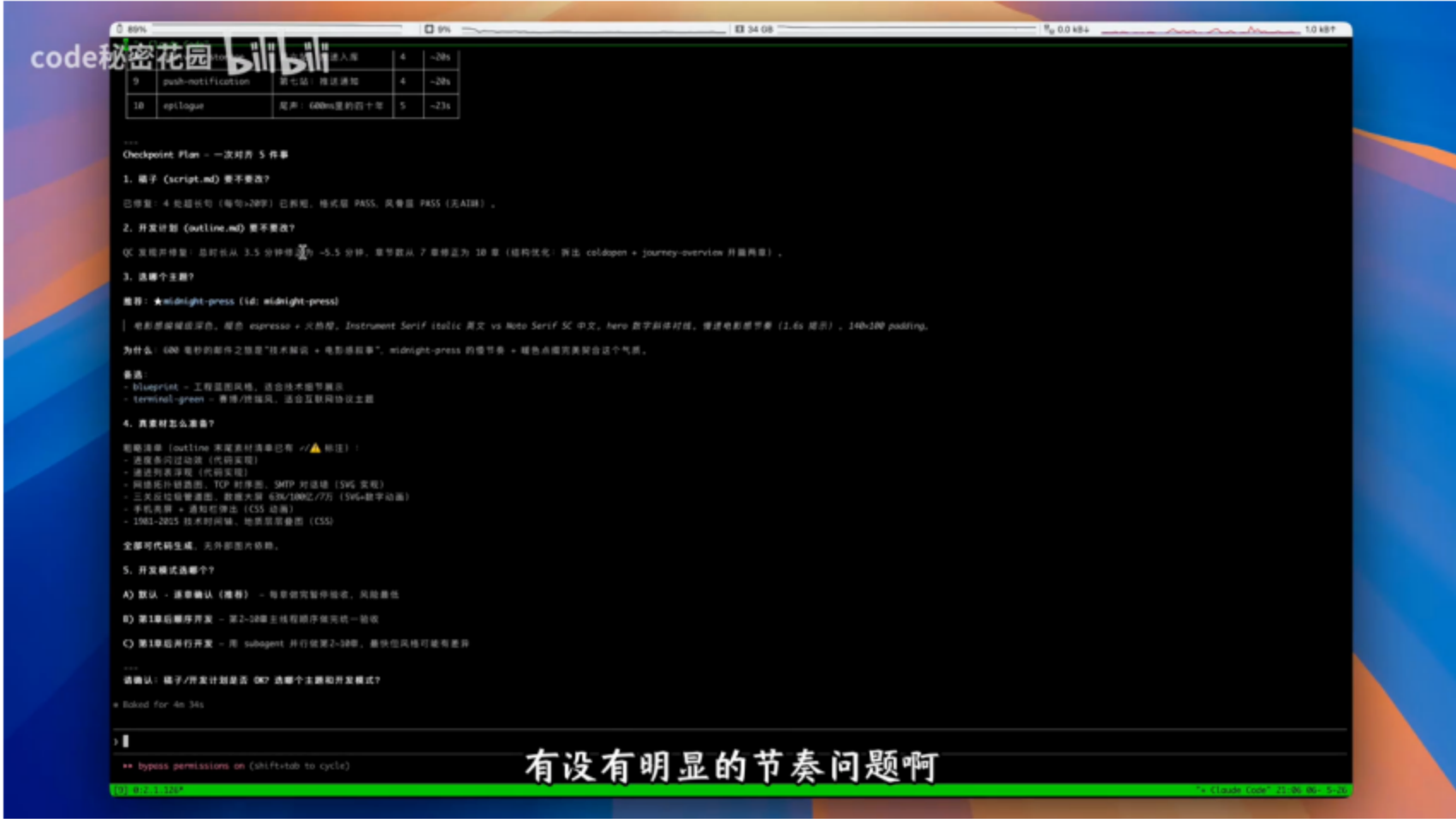


图 14-1 阶段一输出：script.md（口播稿）与 outline.md（开发大纲）

十五、开发阶段：第一章验收与并行开发

内容确认后进入检查点 1（Checkpoint Plan），需要确认：稿子和开发计划是否修改、主题风格、素材准备方式、后续开发模式。

Skill 支持三种开发模式：

- 默认模式：逐章开发，每章完成后人工验收；
- 顺序模式：第一章验收后，后续所有章节按顺序做完再验收；
- 并行模式：第一章验收后，后续章节用 SubAgent 并行开发。

第一章验收特别重要：它决定了页面密度、字体大小、动效节奏、信息呈现方式，是后续所有章节的基准。验收没问题后，再启动 Agent Teams / SubAgent 并行开发剩余章节。

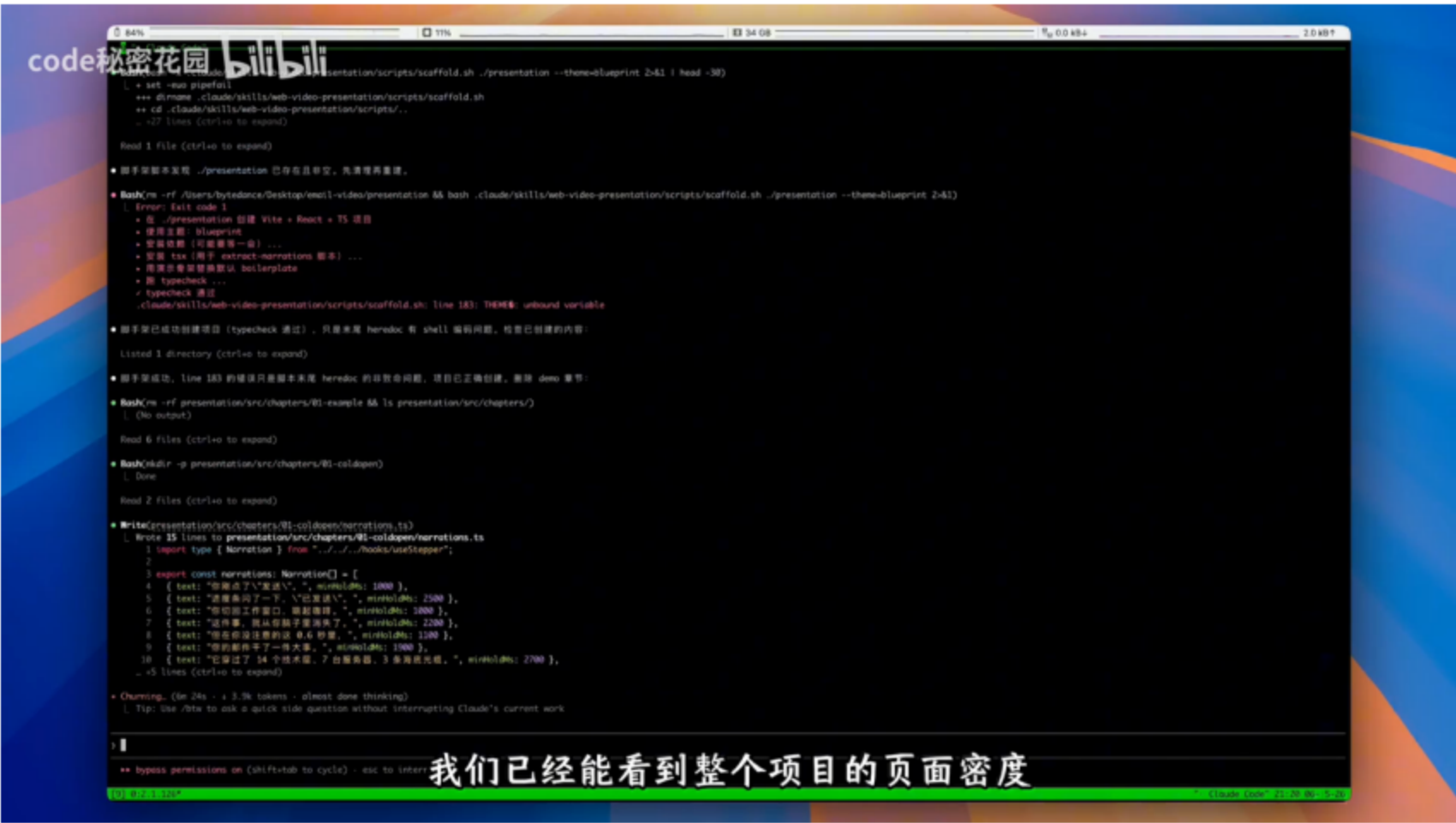


图 15-1 第一章开发完成：项目目录结构搭建起来，narrations.ts 配置每一步的文本和停留时间



图 15-2 全部章节并行开发完成后的交付总览：9 章、56 步、约 7.5 分钟

十六、音频合成与最终网页效果

所有章节开发并质检完成后，进入音频合成阶段。Agent 会从所有章节中抽取出详细的口播文本清单，然后调用 MiniMax CLI 把口播稿批量合成音频。

- 文件名与步骤编号一一对应；
- 已生成文件会自动跳过，中途中断不用重来；
- 遇到速率限制等错误会记录并重新合成。

网页提供了三种录制模式：

- 手动模式：不播放音频，可自由切换步骤，适合真人配音；
- audio=1：每步自动播放音频，但可手动推进；
- auto=1：按下空格后，页面按音频节奏自动播放，直接录屏即可。

因为网页和音频天然对齐，录出来的视频音画同步、效果稳定。

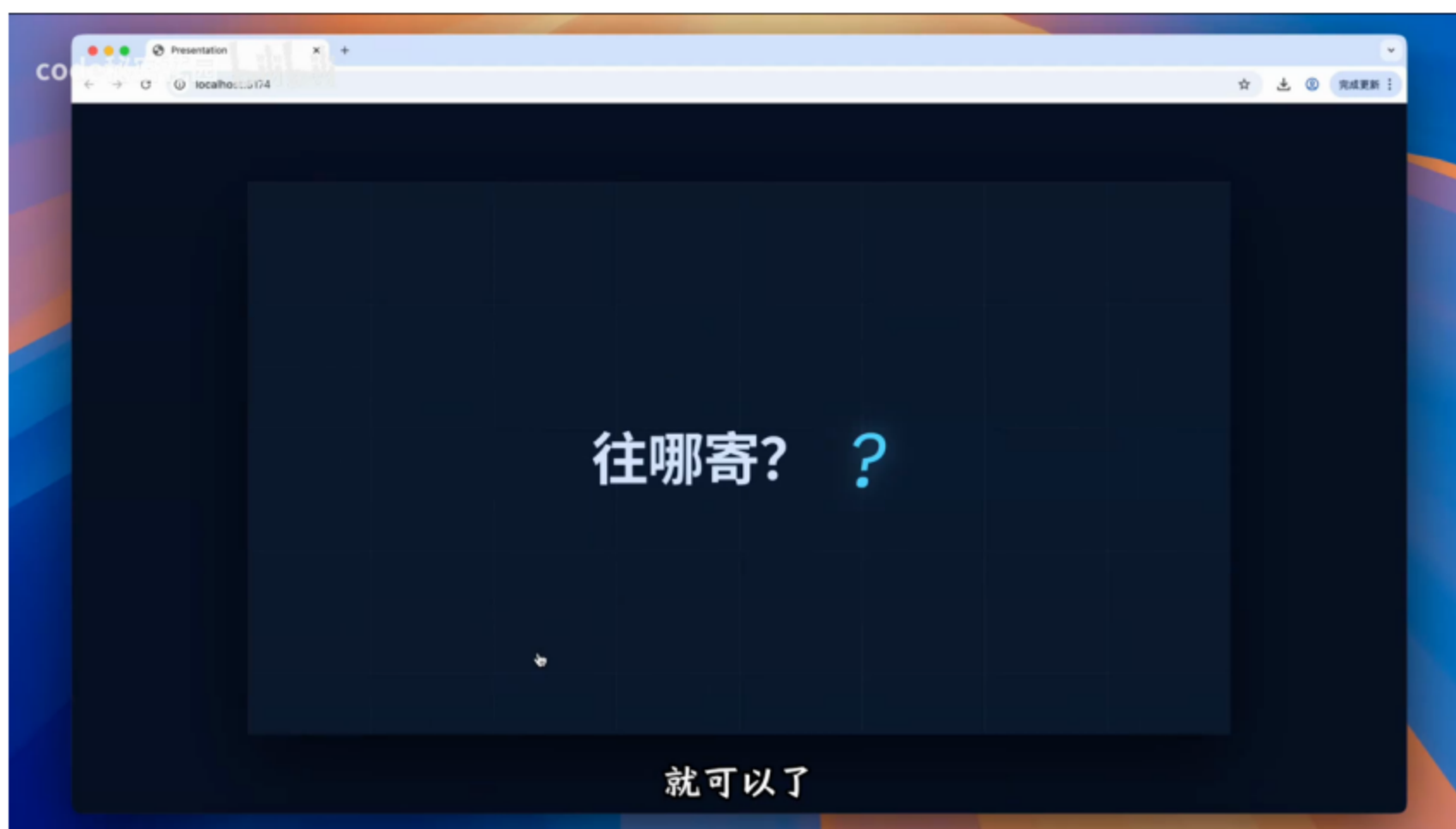


图 16-1 最终生成的网页：按下空格即可按音频节奏自动播放，直接录屏得到成品视频

十七、总结

表面上看，这讲的是一套「文章变成视频」的流程。但本质上，它是一次 Harness 工程的具体实践。

模型本身的能力已经很强：Claude Code 能写代码，MiniMax 提供稳定的推理和语音合成能力。但真正要关注的是：

如何把这些能力稳定地编排起来，让 Agent 能够稳定地完成复杂任务。

这个 Skill 把一次复杂的内容生产任务，拆成了有流程、有状态、有检查点、有自检、有恢复机制的工程系统。Harness 的核心价值，正是把模型能力、工具能力和人的判断，组织成一条稳定可控的生产流程。

Skill 已经开源在 garden-skills 仓库，感兴趣的同学可以拿一篇自己的文章试一试。